

설비 고장을 예측하는 AI 데이터 분석

모델 평가와 해석

혼동행렬 기반 분류 평가

C O N T E N T S

목차

01 정확도와 그 한계

02 혼동행렬

03 정밀도와 재현율

01

정확도와 그 한계

모델 채점의 첫 단추 — 정확도의 정의, 그리고 불균형이 만드는 함정

모델 평가가 필요한 이유

채점을 거쳐야 신뢰할 수 있는 모델 — 평가는 일을 맡기기 전의 면허 시험

01 · 채점

모델은 완성이 아니라
채점을 거친 뒤 신뢰

채점 없이 현장에 붙이면 잘 잡는지 헛경보를
내는지 모르는 채 설비를 맡기는 셈

02 · 면허 시험

일을 맡기기 전 치르는
자격 시험의 자리

면허 없는 사람에게 큰 트럭을 맡기지 않듯, 채
점 안 한 모델에 설비를 맡길 수 없음

03 · 의사결정 언어

채점 결과를 정비
의사결정의 언어로 번역

점수가 아니라 현장에서 어떤 의사결정을 도
울지 말할 수 있는 결과로 변환

평가가 답해 주는 질문들

한 번 보고 끝나는 시험이 아니라, 데이터가 바뀔 때마다 다시 치르는 반복 과정

01 · 채 점

이 모델이 고장을
얼마나 정확히 잡는가

평가가 결국 답해야 하는 단 하나의 질문

02 · 반 복

데이터·환경이 바뀌면
다시 치르는 시험

한 번의 점수로 끝나지 않고 주기적으로 재채
점하는 과정

03 · 역 량

AI를 다루는 내내 쓰는
핵심 능력

분석가의 일상 속 가장 반복되는 작업이자 도구

평가는 정답과 예측을 비교하는 일

모델 평가의 출발점 — 정답지 y_{test} 와 답안지 y_{pred} 의 두 가지 비교

01 · 실제 정답

y_{test}

사람이 이미 아는 결과

모델이 만든 값이 아닌, 채점 기준이 되는 사실

02 · 모델 예측

y_{pred}

모델이 내놓은 답

같은 순서로 같은 설비를 가리켜야 비교가 성립

첫 번째 자리의 정답과 첫 번째 자리의 예측이 같은 설비의 것이어야 채점 성립

정답과 예측 한 칸씩 비교

한 칸씩 정직하게 맞춰 보는 일 — 앞으로 배울 모든 지표의 공통 출발점

CODE · COMPARE.PY

```
# 정답과 예측을 같은 순서로 나란히 비교
y_test = [1, 0, 0, 1, 0]
y_pred = [1, 0, 1, 1, 0]
# 3번째만 틀림 (정상을 고장으로)
```

정답 1·0·0·1·0과 예측 1·0·1·1·0 — 세 번째 설비만 틀림(실제 정상을 고장으로 판단)

정확도(accuracy)란 무엇인가

가장 직관적인 점수 — 시험 100문제 중 몇 개 맞혔나, 그것이 정확도

정확도 공식

$$\text{정확도} = \text{맞힌 개수} \div \text{전체 개수}$$

정상·고장 구분 없이 맞힌 개수를 전체로 나눈 값

전체 중 맞힌 비율

모델이 정답을 맞힌 개수를 전체 개수로 나눈 값 — 분류 평가에서 가장 먼저 떠올리는 숫자

정확도의 성질과 약점

많은 쪽만 맞혀도 점수가 높게 나오는 약점 — 정확도와 데이터 균형을 함께 확인

01 · 범위

0부터 1 사이의 값

보통 백분율로 표기 — 0.93이면 93%로 읽음

02 · 동등 취급

정상·고장을 똑같이 한 개씩 계산

두 클래스 모두 동등하게 한 개로 셈

03 · 약점

불균형 데이터에서 약점이 드러남

많은 쪽만 잘 맞혀도 점수가 높게 나옴

정확도를 볼 때는 항상 데이터가 균형 잡혀 있는지 함께 확인 필요

정확도를 손으로 계산해 보기

5대 중 4대 적중 — 정확도 $4 \div 5 = 0.8$ (80%)

설비	실제	예 측
1번	고장	고장 (맞음)
3번	정상	고장 (틀림)
4번	고장	고장 (맞음)

연습 — 모두 정상이라고만 찍는다면?
실제 정상 3대 맞고 고장 2대 틀려 정확도 $3 \div 5 = 0.6$ — 센서를 보지도 않고 60%

accuracy_score()로 정확도 구하기

정답을 먼저, 예측을 나중에 — 앞으로 모든 지표 함수가 따르는 인자 순서

CODE · ACCURACY_SCORE.PY

```
from sklearn.metrics import accuracy_score
acc = accuracy_score(y_test, y_pred)
print("정확도:", acc)
# 출력: 정확도: 0.8
```

손으로 구한 0.8과 동일한 값 — 함께 따라 칠 코드는 01_혼동행렬_기반_분류_평가_개념코드.py 섹션 5

실습 — 데이터 불러오기와 모델 예측

오늘 모든 평가의 재료가 될 `y_test`와 `y_pred`를 만드는 과정

목 표

C-MAPSS 데이터를 불러와
모델 학습·예측까지 재현

설비 센서 데이터를 불러와 모델을 학습시키고 예측까지 뽑아내는 단계

결 과 물

`y_test`와 `y_pred` 확보

오늘 평가 실습 전체가 여기서 만든 예측을 재료로 사용

실습 — 데이터 로드와 x·y 분리

csv를 불러와 입력 **x**(센서 6종)와 정답 **y**(failure_soon)으로 분리

CODE · LOAD_SPLIT.PY

```
import pandas as pd
df = pd.read_csv("cmapss_fd001_sample.csv")
feature_cols = ["sensor_2", "sensor_3", "sensor_4",
                "sensor_7", "sensor_11", "sensor_15"]
X = df[feature_cols]
y = df["failure_soon"]
```

failure_soon은 RUL이 30 이하일 때 1 — 곧 고장이라는 뜻, 잡아내려는 대상

실습 — 분할·학습·예측

학습용·평가용으로 나눠 RandomForest 학습 후 예측 — 오늘 채점의 재료

CODE · TRAIN_PREDICT.PY

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
Xtr, X_test, ytr, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
model = RandomForestClassifier(random_state=42).fit(Xtr, ytr)
y_pred = model.predict(X_test)
```

random_state=42 고정 — 매번 같은 결과로 다 같이 맞춰 보기 위함

실습 — 정확도 직접 계산해 확인하기

두 가지 방법으로 정확도를 구해 같은 값인지 대조 — 정의를 체화하는 단계

목 표

정확도를 **손으로 구한 뒤**
함수 결과와 대조

방금 만든 예측으로 정확도를 두 가지 방법으로 계산

핵 심

맞은 개수 ÷ 전체
= accuracy_score

직접 센 값과 함수 결과가 일치하는지 확인

실습 — 손계산과 함수 대조

같은 자리의 개수를 직접 세어 전체로 나눈 값과 함수 결과 비교

CODE · VERIFY_ACCURACY.PY

```
correct = (y_test == y_pred).sum()
total = len(y_test)
print("직접 계산:", correct / total)
from sklearn.metrics import accuracy_score
print("함수 결과:", accuracy_score(y_test, y_pred))
```

두 숫자가 정확히 일치하면 정확도가 어떻게 만들어지는지 이해 완료

실습 — 결과 해석 체크포인트

정확도 숫자 하나로 모델을 믿어도 될지 — 정확도의 한계를 의식

01 두 방법의 정확도가 일치하는가
일치했다면 정확도 계산은 완전히 이해

02 이 정확도 숫자만으로 모델을 믿어도 될까
의문을 남겨 두는 단계 — 다음 주제로 연결

03 고장과 정상 중 어느 쪽을 잘 맞힌 결과인지 알 수 있는가
정확도는 어느 쪽을 잘 맞혔는지 구분해 주지 않음 — 한계가 다음 주제로 연결

불균형 데이터란 무엇인가

한 클래스가 다른 클래스보다 훨씬 많은 상태 — 고장 데이터의 자연스러운 모습

01 · 정의

한 클래스가 다른 쪽보다
훨씬 많은 상태

정상 90%·고장 10%처럼 한쪽으로 치우친 데이터

02 · 전형

정상이 대부분,
고장은 소수인 경우

현실의 고장 데이터에서 가장 흔히 만나는 분포

불균형 데이터를 보는 관점

불균형 자체는 현실의 자연스러운 모습 — 진짜 문제는 잘못된 지표로 평가

01 · 자연스러움

불균형 자체는
나쁜 게 **아님**

현실 그대로의 모습 — 없애야 할 결함이 아님

02 · 진짜 문제

잘못된 지표로
평가하는 일

문제는 데이터가 아니라 정확도 같은 부적합
한 지표 선택

03 · 비율 표현

정상 90% : 고장 10%
= **9 대 1**

1%처럼 극단적일수록 평가가 더 까다로움

고장 데이터가 불균형한 이유

설비 수명 대부분은 정상 가동 — 위험한 순간은 수명 끝자락 짧은 구간에만 집중

01 · 수명 분포

대부분 시간은 **정상 가동**
위험은 **수명 끝자락**에 집중

설비는 대부분의 시간을 멀쩡히 가동 — 고장 임박 구간은 짧고 드물

02 · 엔진 200 사이클 예

고장 임박 약 **30 사이클**
한 대당 **15%**만 임박, 85% 정상

RUL 30 이하 구간만 양성 라벨 — 자연스럽게 매우 불균형

불균형은 고장 예측의 기본값

거의 모든 고장 예측 데이터에 공통된 성질 — 처음부터 불균형에 강한 평가 준비

01 · 공통 성질

거의 모든 고장 예측 데이터
불균형이 기본

우리 데이터만의 특성이 아니라 분야 공통 성질

02 · 준비 자세

처음부터 불균형에 강한
평가 방법을 준비

failure_soon 기준을 짧게 잡을수록 더 불균형 — 라벨 설계도 평가에 영향

정확도의 함정

고장을 단 하나도 못 잡아도 정확도는 95%로 나올 수 있는 충격적 사실

함정의 정체

정상 95% 데이터에서 무조건 정상이라 찍으면 정확도 95%

모델이 무조건 정상이라고만 찍어도 점수가 95점으로 나오는 구조

고장을 하나도 못 잡아도 점수는 높게 나옴

정작 잡고 싶은 고장은 모두 놓쳤는데 95점 — 이것이 정확도의 함정

정확도의 함정이 위험한 이유

점수가 나빠 보이지 않아 더 위험 — 높은 점수가 오히려 위험을 가림

01 · 착시

높은 점수가
좋은 모델이라는 착각

30%였다면 다들 이상하다고 다시 봤겠지만, 95%는 그렇지 않음

02 · 방심

95%면 안심하고
그냥 넘어가 버림

사람은 자연스럽게 안심 — 높은 점수가 오히려 위험을 가림

불균형 데이터에서 정확도가 높을수록, 혹시 함정인지 한 번 더 의심

베이스라인(더미 분류기)과 비교하기

정확도가 잘한 건지 판단하려면 **기준선**이 필요 — 그것이 베이스라인

01 · 정의

아무것도 학습 안 한
기준선 모델

학습 없이도 만들 수 있는 가장 간단한 비교 대상

02 · 동작 방식

무조건 **가장 많은 클래스**
로만 찍는 게으른 모델

데이터를 보지도 않고 다수 클래스만 출력

03 · 판단 기준

의미 있게 높아야
진짜 학습한 것

베이스라인보다 의미 있게 높아야 모델이 학습한 것

더미 분류기로 기준선 만들기

`most_frequent` 전략 — 항상 가장 많은 클래스로만 예측하는 모델

CODE · DUMMY_BASELINE.PY

```
from sklearn.dummy import DummyClassifier
dummy = DummyClassifier(strategy="most_frequent")
dummy.fit(Xtr, ytr)
print("베이스라인:", dummy.score(X_test, y_test))
```

정상 비율 85% → 더미도 정확도 85% · 우리 모델 86%면 1%p 차이로 학습 거의 안 됨 · 95%면 10%p 차이로 의미 있는 학습

실습 — 불균형 분포 확인과 함정 체험

데이터의 정상·고장 비율을 확인하고 함정을 직접 체험하는 두 단계 실습

목 표

데이터의 정상·고장 비율
확인 후 함정 직접 체험

우리 데이터가 얼마나 불균형한지 눈으로 확인

핵심

무조건 정상 예측의
정확도 측정

무조건 정상이라 찍었을 때 점수가 얼마나 높게 나오는지 체험

실습 — 클래스 비율 확인

정상(0)과 고장 임박(1)의 개수·비율 확인 — 1이 얼마나 적은지 눈으로 확인

CODE · CLASS_RATIO.PY

```
print(y.value_counts())  
print(y.value_counts(normalize=True))  
# 0(정상)이 다수, 1(고장임박)이 소수
```

value_counts로 개수 · normalize=True로 비율 확인 — 고장 임박(1) 비율이 생각보다 적음

실습 — 무조건 정상 예측의 정확도

모두 정상으로 찍어도 정확도가 꽤 높음 — 함정을 눈으로 확인하는 순간

CODE · ALL_NORMAL.PY

```
import numpy as np
all_normal = np.zeros(len(y_test))
from sklearn.metrics import accuracy_score
print("무조건 정상:", accuracy_score(y_test, all_normal))
```

모든 예측을 0(정상)으로 만들고 정확도 측정 — 고장을 하나도 안 잡았는데도 꽤 높은 정확도

실습 — 함정 체험 정리

점수는 높지만 고장은 0개 — 점수와 실제 쓸모가 완전히 따로 노는 순간

01 무조건 정상 예측의 정확도가 몇 %인가
숫자를 적어 두고 정상 비율과 거의 같은지 확인

02 그 모델이 고장을 몇 개나 잡았는가 (**0개**)
정확도는 높지만 고장은 0개 — 점수와 쓸모가 따로 노는 상태

03 정확도만 보면 왜 위험한지 한 문장으로 정리
"정확도 하나만 보면 고장을 다 놓친 모델도 좋아 보인다" 한 문장 정리

실습 — 베이스라인 대비 모델 성능 비교

더미 분류기와 우리 RandomForest의 정확도를 나란히 — 차이가 의미 있는 수준인지 판단

목 표

더미 분류기와
우리 모델 **정확도 비교**

더미 분류기와 우리 RandomForest의 정확도를 나란히 두고 비교

핵 심

차이가 **의미 있는**
수준인지 판단

우리 모델이 정말로 베이스라인을 의미 있게 넘었는지 판단

실습 — 두 모델 정확도 출력

더미와 모델의 정확도를 나란히 출력 — 두 숫자를 비교할 준비

CODE · TWO_ACCURACIES.PY

```
dummy_acc = dummy.score(X_test, y_test)
model_acc = accuracy_score(y_test, y_pred)
print("베이스라인:", dummy_acc)
print("우리 모델:", model_acc)
```

앞에서 만든 더미와 우리 모델의 정확도를 나란히 출력 — 비교 준비

실습 — 차이 계산과 판단

두 정확도의 차이를 퍼센트포인트로 — 1~2%p면 학습 거의 안 됨, 10%p 이상이면 의미 있음

CODE · ACCURACY_GAP.PY

```
gap = model_acc - dummy_acc
print("차이(%p):", round(gap * 100, 2))
# 차이가 클수록 의미 있는 학습
```

이 차이가 클수록 모델이 베이스라인을 넘어 진짜 학습한 것

실습 — 베이스라인 비교 정리

정확도 옆에는 항상 **기준선**이 필요 — 점수 하나만으로는 해석 불가

01 **우리 모델이 베이스라인보다 높은가**
더미보다 정확도가 높은지 단순 비교

02 **그 차이가 의미 있는 수준인가**
1~2%p와 10%p 이상은 의미가 완전히 다름

03 **정확도 옆에는 항상 기준선이 필요하다는 결론 정리**
점수 하나로는 해석 불가 — 베이스라인과 함께 봐야 잘한 건지 판단 가능

02

혼동행렬

네 칸짜리 성적표 — TP·FN·FP·TN과 설비 비용의 언어

평가 지표가 여러 개 필요한 이유

정확도는 **총점** 하나일 뿐 — 같은 점수라도 모델의 성격은 완전히 다를 수 있음

01 · 총점의 한계

정확도는 **총점** 하나일 뿐
어떤 실수인지 못 가림

전체에서 몇 개 맞았나만 알려 주는 총점 — 어떤 종류의 실수를 했는지는
가려 주지 못함

02 · 모델의 성격

같은 점수라도
모델의 성격은 **완전히 다름**

실수의 종류를 가려 주는 지표가 별도로 필요

총점과 과목별 점수의 비유

수학 100·영어 0과 둘 다 50은 총점이 같아도 완전히 다른 학생

01 · 총점

전체 수준만 알고
강·약점은 안 보임

어느 과목을 잘하고 못하는지 가릴 수 없음

02 · 과목별

수학 100·영어 0과
둘 다 50은 전혀 다름

총점이 같아도 학습 상태는 완전히 다름

03 · 모델

고장 잘 잡음과
헛경보 적음은 다름

정확도가 같아도 두 가지 능력은 따로 봐야 함

문제별 핵심 실수에 맞는 지표

가장 중요한 실수를 정하고 그 실수를 비추는 지표 선택

원칙

지표가 많다고 무조건 좋은 건 아님 — **핵심 실수**를 비추는 지표 선택

이 문제에서 가장 중요한 실수를 정하고 그 실수를 잘 비춰 주는 지표를 고르는 일

우리 문제

고장 예측의 핵심 실수 = **고장을 놓침** → **재현율**이 핵심 지표

놓침을 반영하는 재현율 — 정밀도와 함께 짝으로 추적

양성과 음성을 정하는 기준

양성은 많은 쪽이 아니라 **찾아내려는 쪽** — 우리에게겐 고장 임박이 양성

01 · 양성 (Positive)

우리가 **찾아내려는 것**
= 고장 임박

의료에서 건강한 사람이 많아도 질병을 양성으로 두는 것과 동일한 원리

02 · 음성 (Negative)

찾아내려는 게 아닌 쪽
= **정상**

수가 많은 다수 클래스이자, 양성의 반대편

양성을 거꾸로 잡는 실수

초보자가 자주 하는 실수 — 많은 쪽(정상)을 양성으로 두는 경우

구분	결과
✖ 정상을 양성으로	모든 지표가 정상을 얼마나 잘 맞혔나 를 재게 됨 — 정작 궁금한 고장 포착력을 놓침
✔ 고장 임박을 양성으로	양성은 찾아내려는 쪽 — 모든 지표가 고장 포착력 을 비추도록 정렬
양성은 많은 쪽이 아니라 찾아내려는 쪽 — 머릿속에 단단히 새기기	

혼동행렬의 네 칸

실제(행)와 예측(열)을 교차시킨 네 칸짜리 표 — 모든 평가 지표의 출발점

구분	예측 정상	예측 고장
실제 정상	TN 정상을 정상으로	FP 정상을 고장으로 (헛경보)
실제 고장	FN 고장을 놓침 (가장 위험)	TP 고장을 고장으로

행은 실제, 열은 예측 — 0(정상)이 앞에 와서 왼쪽 위가 TN, 오른쪽 아래가 TP

네 칸 이름 읽는 규칙

뒷글자는 **모델의 예측**, 앞글자는 **그 예측이 맞았는지**

TP

고장을 고장으로 맞힘

예측: 양성 / 결과: 맞음 → 진짜 잡아낸 고장

FN

고장을 정상으로 놓침 (가장 위험)

예측: 음성(정상) / 결과: 틀림 → 모델이 정상이라 판정한 것이 틀린 고장

FP

정상을 고장으로 잘못 경보

예측: 양성(고장) / 결과: 틀림 → 헛경보

TN

정상을 정상으로 맞힘

예측: 음성(정상) / 결과: 맞음 → 조용한 정상 판정

규칙만 알면 네 칸을 외울 필요 없이 그때그때 풀어 읽기 가능

네 칸의 성질과 기억법

FN(놓친 고장)은 언제나 가장 먼저 보는 칸 — 모든 지표는 네 칸을 다르게 나눈 비율

01 · 분포

모든 설비는 네 칸 중
정확히 한 칸에 들어감

네 칸의 합 = 평가한 전체 설비 수

02 · 대각선

TP·TN = 맞힌 칸

FP·FN = 틀린 칸

대각선에 맞힌 칸, 반대 대각선에 틀린 칸

03 · 가장 먼저

FN(놓친 고장)은
언제나 가장 먼저 보는 칸

TP 초록·TN 회색·FP 노랑·FN 빨강 — 빨간 FN이
가장 먼저 눈에 들어가야 할 칸

TP — 고장을 고장으로 맞춤

진짜 위험을 진짜 위험으로 알아본 순간 — 고장 예측 모델의 존재 이유

01 · 정의

실제 고장을
고장이라고 맞힌 경우

모델이 제값을 하는 순간 — 정비팀이 미리 손쓸 기회 제공

02 · 예시

실제 고장 40대 중
34대 맞춤 → TP = 34

이 34대는 모두 정비팀이 미리 손쓸 기회를 얻은 설비

TP가 거의 없다면 고장 예측 모델로서 존재 의미가 없음 — 혼동행렬에서 TP가 충분히 큰지 먼저 확인

FN — 고장을 놓침

가장 중요한 칸 — 위험을 놓친 경우, 정비 현장에서 **가장 비싼 실수**

FN

실제 고장을 정상으로 놓친 경우

실제로 고장 임박인데 모델이 정상이라고 판단 — 위험을 놓친 결과

정비 현장에서 **가장 비싼 실수**

설비 정지로 직결되는 손실 — FN 한 건의 비용이 다른 실수보다 훨씬 큼

FN이 가장 비싼 이유

FN = 아무 경고 없이 갑자기 멈춘 설비 — 같은 고장이라도 손실이 완전히 다름

케이스	결과
-----	----

✓ 미리 알고 계획 정비 (TP)	부품만 교체로 끝남 — 정비팀이 일정 안에서 대응 가능
--------------------	--------------------------------

✗ 모르고 당한 정지 (FN)	연쇄 손상 + 긴 복구 — 비용이 몇 배로 불어남
------------------	-----------------------------

FN을 가장 먼저, 가장 신중하게 — 정비팀은 위험한지조차 모른 채 대비 불가

FP — 정상을 고장으로 잘못 경고

멀쩡한 설비를 고장이라 헛경보한 경우 — 점검 헛수고와 경보 신뢰 저하

01 · 정의

멀쩡한 설비를
고장이라 **헛경보**한 경우

실제로는 정상인데 모델이 고장이라고 잘못 외친 경우

02 · 결과

점검 헛수고 발생,
경보 신뢰 저하

잘못된 헛경보는 정비팀이 경보를 무시하게 만드는 부작용

FP의 비용과 관리

FP는 없앨 것이 아니라 **적정 수준**으로 관리할 것

01 · 예시

경보 50대 중
12대 멀쩡 → **FP = 12**

점검 시간·인력을 썼지만 문제가 없었던 경우

02 · 주의

FP를 0으로만 만들려 하면
모델이 신중해져 **FN 증가**

헛경보 줄이려다 진짜 고장 놓치는 함정 — 적정 수준으로 관리

TN — 정상을 정상으로 맞춤

수가 많고 쉽게 얻어져 우열 판단의 비중은 낮음 — 그래도 조용한 기여

01 · 정의

멀쩡한 설비를
멀쩡하다고 맞힌 경우

실제 정상을 정상이라 맞힌 칸 — 네 칸 중 가장
흔함

02 · 비중 낮음

수가 많고 쉽게 얻어져
우열 판단에서는 비중↓

불균형 데이터에선 TN이 워낙 많아 모델 차이
를 가리지 못함

03 · 조용한 기여

괜한 점검을 막아 주는
조용한 기여

TN 없이는 모든 설비를 고장이라 외치는 쓸모
없는 경보기

모델 우열은 TP·FN·FP에 더 집중해서 판단

혼동행렬 표 읽는 법

행=실제, 열=예측, 0(정상)이 먼저 — 왼쪽 위 TN, 오른쪽 아래 TP

구분	예측 0 (정상)	예측 1 (고장)
실제 0 (정상)	TN 왼쪽 위	FP 오른쪽 위
실제 1 (고장)	FN 왼쪽 아래	TP 오른쪽 아래

위치만 익히면 어떤 혼동행렬을 봐도 곧바로 의미 파악 가능

혼동행렬 읽기 연습

첫째 줄 95, 5 / 둘째 줄 8, 32 — 위치만으로 네 칸 의미 즉시 해석

예시 혼동행렬

**[[95 5]
[8 32]]**

실제 정상 100대

95 정상으로 맞춤 (TN) · 5 고장으로 잘못 봄 (FP)

실제 고장 40대

8 놓침 (FN) · 32 잡음 (TP)

위치만으로 네 칸 의미를 바로 읽는 연습

연습이 되어 있으면 코드 결과도 막힘없이 해석 가능

confusion_matrix()와 출력 순서 주의

ravel로 펼치면 순서대로 **tn, fp, fn, tp** — 이 순서를 헷갈리면 모든 지표가 엉터리

CODE · CONFUSION_MATRIX.PY

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()
print(tn, fp, fn, tp)
```

confusion_matrix는 2x2 배열 반환 — ravel로 펼치면 $tn \rightarrow fp \rightarrow fn \rightarrow tp$ 순서로 정렬

출력 순서가 정해지는 이유

sklearn이 라벨을 **오름차순**(0 먼저, 1 나중) 정렬하기 때문

01 · 정렬 규칙

sklearn은 라벨을
오름차순으로 배치

0이 먼저, 1이 나중 — 왼쪽 위 TN, 오른쪽 아래 TP로 자연스럽게 정렬

02 · 활용

순서는 **tn, fp, fn, tp**
그대로 믿고 사용

라벨이 0, 1이 아니거나 순서를 바꾸려면 labels 옵션으로 지정

혼동행렬 시각화 (heatmap)

ConfusionMatrixDisplay로 색칠된 표 생성 — 진한 칸이 많은 칸

CODE · HEATMAP.PY

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt
ConfusionMatrixDisplay(cm).plot()
plt.show()
```

시각화는 설명에도 강력 — 색칠된 그림으로 위험 지점을 손가락으로 짚어 보이기 가능

설비 관점에서 FN과 FP의 비용 차이

혼동행렬을 단순한 점수가 아니라 **비용**으로 읽기

01 · 본질

FN(놓침)과 FP(헛경보)는
비용이 다름

같은 한 건의 실수라도 손실 규모는 칸마다 완전히 다름

02 · 일반 경향

대체로 **FN이 FP보다**
훨씬 비쌈

FN은 설비 정지로 이어지고, FP는 점검 한 번의 헛수고

FN과 FP 비용 비교

실수 한 건당 비용을 따져 보면 어느 실수를 줄일지 자연스럽게 드러남

실수	의미	대략 비용
FN	설비 정지 한 건 — 아무 경고 없이 멈춤	매우 큼
FP	점검 한 번 헛수고 — 출동했더니 멀쩡함	상대적 작음

분해 점검이 매우 비싼 현장이라면 FP가 더 클 수도 있음 — 우리 현장은 현장 담당자와 함께 확인

비용을 한 문장으로 정리

성능을 **금액의 언어**로 말하면 현장이 알아들음

한 문장으로

"이 모델이 틀릴 때, **그 한 번이 얼마짜리 실수인가**"

모델이 한 번 틀릴 때의 비용으로 환산해 보는 사고 습관

성능을 **금액의 언어**로 — 경영진과 현장이 알아듣는 언어

네 칸을 금액으로 환산하면 "몇 % 맞았다"가 아니라 "얼마를 아꼈고 얼마를 잃을 위험"으로 전달 가능

실습 — 혼동행렬 손으로 만들어 보기

코드 전에 손으로 — 작은 예시로 각 설비가 어느 칸인지 직접 분류

목 표

작은 예시 데이터로
네 칸을 직접 채움

각 설비가 TP·FN·FP·TN 중 어디인지 손으로 분류

핵심

각 설비가
어느 칸에 들어가는지 분류

앞·뒷글자 규칙으로 한 칸씩 짝어 가기

실습 — 예시로 네 칸 채우기

여섯 대를 한 칸씩 분류 → TP=2, FN=1, FP=1, TN=2 — 함수 결과와 일치 확인

CODE · CM_BY_HAND.PY

```
y_true = [1, 0, 1, 1, 0, 0]
y_hat = [1, 0, 0, 1, 1, 0]
# 직접 분류: TP=2, FN=1, FP=1, TN=2
from sklearn.metrics import confusion_matrix
print(confusion_matrix(y_true, y_hat).ravel())
```

첫째 TP · 셋째 FN · 다섯째 FP — 직접 센 값과 confusion_matrix 결과가 같은지 확인

실습 — 손계산 체크포인트

직접 분류한 결과와 함수 결과가 일치 — 혼동행렬 완전 이해의 신호

01 직접 분류한 결과와 함수 결과가 일치하는가
일치한다면 혼동행렬을 완전히 이해

02 FN과 FP가 각각 몇 개인지 정확히 찾았는가
앞글자·뒷글자 규칙으로 FN과 FP를 헛갈리지 않고 찾기

03 네 칸의 합이 전체 설비 수와 같은가
 $TP + FN + FP + TN = \text{전체 평가 설비 수}$

실습 — confusion_matrix 생성과 시각화

실제 모델 예측으로 혼동행렬 생성·시각화 — 네 칸 숫자와 heatmap 그림 확보

목 표

실제 모델 예측으로
혼동행렬 생성·시각화

앞에서 만든 y_test, y_pred로 진짜 혼동행렬 작성

결 과 물

네 칸 숫자와
heatmap 그림

숫자 출력 + 색칠된 그림 두 가지를 모두 확보

실습 — 혼동행렬 생성

우리 모델 예측으로 혼동행렬 출력 — 2×2 배열 확인

CODE · CM_GEN.PY

```
from sklearn.metrics import confusion_matrix  
cm = confusion_matrix(y_test, y_pred)  
print(cm)
```

2×2 배열 출력 — 왼쪽 위가 TN, 오른쪽 아래가 TP 위치 확인

실습 — heatmap 시각화

진한 칸이 어디인지 눈으로 확인 — **FN** 칸이 진하면 놓침 많다는 신호

CODE · CM_HEATMAP.PY

```
from sklearn.metrics import ConfusionMatrixDisplay
import matplotlib.pyplot as plt
ConfusionMatrixDisplay(cm).plot()
plt.show()
```

그림에서 위험 지점을 바로 찾는 연습 — FN 칸이 진하다면 놓침이 많다는 신호

실습 — 시각화 체크포인트

FN 칸의 숫자가 우리 모델이 놓친 고장의 수 — 꼭 확인할 핵심 숫자

01 혼동행렬의 네 칸 위치를 정확히 읽었는가
왼쪽 위 TN, 왼쪽 아래 FN, 오른쪽 위 FP, 오른쪽 아래 TP

02 FN 칸의 숫자가 몇인지 (**놓친 고장 수**)
우리 모델이 놓친 고장의 수 — 가장 중요한 숫자

03 그림에서 가장 진한 칸이 어디인가
위험 지점을 시각적으로 빠르게 파악

실습 — TP/FP/FN/TN 추출과 설비 해석

숫자를 **현장 언어**로 번역 — 각 칸을 "설비 몇 대"로 말하기

목 표

네 칸을 변수로 추출해
설비 대수로 해석

네 칸을 꺼내 "설비 몇 대"라는 현장 언어로 표현

핵 심

숫자를
현장 언어로 번역

분석가의 핵심 능력 — 숫자를 의사결정 언어로 바꾸는 일

실습 — 네 칸 추출

ravel로 네 칸을 각 변수에 담기 — 순서 **tn, fp, fn, tp** 엄수

CODE · EXTRACT_FOUR.PY

```
tn, fp, fn, tp = cm.ravel()  
print("TP:", tp, "FN:", fn)  
print("FP:", fp, "TN:", tn)
```

각 변수가 설비 몇 대를 뜻하는지 말할 수 있는 단계 — 순서를 꼭 지킬 것

실습 — 설비 대수로 해석 출력

각 칸을 현장 언어 문장으로 — 숫자가 현장의 의미로 바뀌는 순간

CODE · INTERPRET.PY

```
print(f"잡은 고장 {tp}대, 놓친 고장 {fn}대")  
print(f"헛경보 {fp}대, 정상 확인 {tn}대")
```

잡은 고장, 놓친 고장, 헛경보, 정상 확인 — 네 칸이 의사결정 언어로 변환

실습 — 설비 해석 체크포인트

"이 모델은 고장 N대를 놓쳤다" 문장 완성 — 혼동행렬 파트 완전 익힘의 신호

01 네 칸을 올바른 순서로 추출했는가

tn, fp, fn, tp = cm.ravel() — 순서 엄수

02 놓친 고장(FN)을 설비 대수로 말할 수 있는가

숫자가 아니라 "고장 N대"라는 의사결정 언어로 표현

03 "이 모델은 고장 N대를 놓쳤다" 문장 완성

숫자를 현장 언어로 번역하는 분석가의 핵심 능력 체득

03

정밀도와 재현율

경보의 신뢰도와 놓치지 않는 능력 — 두 지표의 짝과 트레이드오프

비율 지표가 필요한 이유

절대 개수만으로는 공정한 비교 불가 — **비율**로 바뀌야 같은 잣대 가능

01 · 절대 개수의 한계

혼동행렬의 네 칸은
절대 개수

공장 규모가 다르면 단순 비교 불가

02 · 두 공장 비교

큰 공장 80건 vs
작은 공장 8건

절대 수만 보면 10배 좋아 보이지만 전체 고장
수가 다름

03 · 비율로 전환

전체 중 **몇 %**를 잡았나
= 공정한 비교

그 비율이 정밀도와 재현율

정밀도(precision)란 무엇인가

모델이 고장이라 한 것 중 진짜 고장의 비율 — **경보 중 진짜의 비율**

정밀도 공식

$$\text{정밀도} = \text{TP} \div (\text{TP} + \text{FP})$$

분모는 모델이 고장이라 외친 전체 — TP(진짜 고장) + FP(헛경보)

고장이라 한 것 중 진짜 고장의 비율

모델이 고장이라 외친 것 중 진짜였던 비율 — 경보의 정확도

정밀도 외우는 법

정밀도 = 내가 한 말 중에 진짜였던 비율

01 · 외우는 문장

내가 한 말 중에
진짜였던 비율

모델이 고장이라 외친 것 중 진짜였던 비율

02 · 높을 때

높으면 모델이
신중함

입이 무거워 함부로 고장이라 외치지 않음

03 · 낮을 때

낮으면 아무 데나
고장이라 떠벌림

분모 0일 땐 계산 불가 — sklearn은 경고 후 0 처리

정밀도의 의미 — 경보의 신뢰도

점검이 비싼 정밀가공 라인 — 거짓 경보 한 번이 큰 손실, 정밀도가 특히 중요

정밀도

현장 의미

✓ 높으면

모델이 고장이라 하면 대체로 진짜 → 믿고 출동

✗ 낮으면

헛걸음 잦음 → 정비팀이 경보를 무시하기 시작 (양치기 소년)

정밀가공 라인처럼 점검이 비싸면 정밀도가 특히 중요, 점검이 저렴하면 좀 낮아도 감당 가능

precision_score() 사용법

정답을 먼저, 예측을 나중에 — 라벨 0,1이면 양성 1 기준으로 자동 계산

CODE · PRECISION_SCORE.PY

```
from sklearn.metrics import precision_score
p = precision_score(y_test, y_pred)
print("정밀도:", round(p, 4))
# 양성(1) 기준, 분모 0이면 경고 후 0
```

분모 0(고장 예측 한 번도 없음)일 땐 정밀도 숫자보다 왜 그랬는지 먼저 살피기

재현율(recall)이란 무엇인가

우리 고장 예측에서 **가장 중요한 지표** — 실제 고장 중 잡아낸 비율

재현율 공식

$$\text{재현율} = \text{TP} \div (\text{TP} + \text{FN})$$

분모에 **FN(놓친 고장)** — 놓침이 많을수록 재현율 하락

실제 고장 중 모델이 잡아낸 비율

분모는 실제 고장 전체 — TP(잡은 고장) + FN(놓친 고장)

재현율 외우는 법

재현율 = 실제 일어난 일 중 내가 잡아낸 비율 (포착률)

01 · 외우는 문장

실제 일어난 일 중
내가 잡아낸 비율

포착률이라고도 부름 — 그물의 촘촘함

02 · 비유

정밀도 = 말의 정확함
재현율 = 그물의 촘촘함

그물이 촘촘할수록 빠져나가는 고장이 줄어들

03 · 예시

재현율 0.9
= 10건 중 9건 잡고 1건 놓침

현장 위험과 곧바로 연결되는 직관적 숫자

재현율의 의미 — 놓치지 않는 능력

고장 예측은 정확도보다 **재현율**을 먼저 — 놓치면 큰일 나는 문제의 핵심 지표

01 · 본질

재현율 =
놓치지 않는 능력

얼마나 잘 잡아냈는지를 보는 지표

02 · 핵심 도메인

고장·질병·사기처럼
놓치면 **큰일** 나는 문제

공통적으로 재현율을 우선하는 분야

03 · 위험 신호

정확도 높은데 재현율 낮음
→ **위험한 모델**

고장 예측은 정확도보다 재현율을 먼저

recall_score() 사용법

사용법은 precision_score와 완전히 동일 — 이름만 다름

CODE · RECALL_SCORE.PY

```
from sklearn.metrics import recall_score
r = recall_score(y_test, y_pred)
print("재현율:", round(r, 4))
# 정답 먼저, 예측 나중 - 순서 동일
```

실무에서는 정밀도·재현율을 따로 구하지 않고 한 번에 같이 출력 — 짝으로 봐야 의미 있음

정밀도와 재현율 함께 보기

정밀도·재현율은 **한 쌍의 안경** — 한쪽만 보면 왜곡

01 · 한 쌍의 안경

한쪽 알로만 보면
세상이 **왜곡**되어 보임

두 숫자를 나란히 두고 함께 읽기

02 · 함께 읽는 법

경보의 **신뢰도**와
놓치지 않는 능력

classification_report로 한 번에 출력, F1-score는 둘을 합친 값

정밀도-재현율 트레이드오프

정밀도와 재현율은 보통 한쪽이 오르면 한쪽이 내리는 **시소** 관계

시소 관계

한쪽이 오르면 한쪽이 내림 — 둘 다 완벽은 환상

적극적으로 잡으면 헛경보 증가, 신중하면 놓침 증가 — 비용 기준으로 한쪽 선택

헛경보 줄이려 신중해지면 → 놓침 증가 (재현율 ↓)

고장을 더 적극적으로 잡으면 → 헛경보 증가 (정밀도 ↓)

적극적 vs 신중한 모델

어느 쪽을 택할지는 어느 실수가 더 비싼지로 결정

01 · 적극적

의심되면 다 경보
재현율↑ 정밀도↓

조금만 의심돼도 경보 → 놓침 ↓ · 헛경보 ↑

02 · 신중함

확실할 때만 경보
정밀도↑ 재현율↓

확실할 때만 경보 → 헛경보 ↓ · 놓침 ↑

03 · 선택

어느 실수가
더 비싼지로 결정

비용 기준의 의식적 선택 — 점수 최적화가 아닌 비용 최적화

트레이드오프를 보는 눈 — 공항 보안 비유

보안이 까다로움을 택하는 건 **놓침이 너무 비싸기 때문**

방식	결과
----	----

까다롭게	위험물 놓침 ↓ (재현율↑) · 멀쩡한 승객 붙잡음 ↑ (정밀도↓)
-------------	---

느슨하게	줄 빠름 (정밀도↑) · 위험물 놓침 ↑ (재현율↓)
-------------	---

둘 다 완벽이라는 환상을 버리고 비용 기준으로 의식적으로 선택

설비 정비에서의 우선순위 — 세 단계

더 큰 손실을 줄이는 지표를 우선 — 대개는 **재현율**

STEP 1

손실 계산

놓침(FN)과 헛경보(FP)의 손실을 각각 계산



STEP 2

크기 비교

둘 중 어느 손실이 더 큰지 비교



STEP 3

지표 선택

더 큰 손실을 줄이는 지표 우선 (대개 재현율)

우선순위를 정한 뒤

재현율 우선 = **먼저, 가장 무겁게** — 정밀도 무시하는 아님

01 · 우선의 정의

재현율을 먼저, 가장
무겁게 본다는 뜻

정밀도를 무시한다는 뜻이 아님

02 · 동반 감시

정밀도가 감당 못 할 만큼
무너지지 않게 감시

실무에서는 두 지표를 항상 같이 봄

우선순위 판단의 핵심 태도

점수 높이기가 아니라 **비용 논리에 근거한 방향 결정**

핵심 태도

가장 비싼 실수를 먼저 줄이되, 다른 비용 폭발도 함께 감시

가장 비싼 실수를 가장 먼저 줄이되 그 과정에서 다른 비용이 폭발하지 않는지 함께 확인

비용 논리에 근거한 방향 결정 — 정비 의사결정의 핵심

막연히 점수 높이기가 아니라, 비용 논리에 근거해 방향을 정하는 것 — 데이터 분석의 핵심 태도

실습 — 정밀도·재현율 손계산과 sklearn 확인

네 칸으로 정밀도·재현율 손계산 후 함수와 대조 — 공식 일치 확인

목 표

네 칸으로 정밀도·재현율
손계산 후 함수와 대조

혼동행렬의 네 칸으로 두 지표를 직접 계산

핵 심

공식과 함수 결과
일치 확인

손계산과 sklearn 결과가 같다면 공식을 완전히 이해

실습 — 네 칸으로 손계산

공식에 네 칸을 직접 대입 — 정밀도 = $TP/(TP+FP)$, 재현율 = $TP/(TP+FN)$

CODE · PR_HANDCALC.PY

```
tn, fp, fn, tp = cm.ravel()
precision = tp / (tp + fp)
recall = tp / (tp + fn)
print("정밀도:", round(precision, 4))
print("재현율:", round(recall, 4))
```

공식을 손으로 써 보며 분모 차이를 익히기

실습 — 함수와 대조

손계산과 함수 결과 비교 — 일치하면 공식을 완전히 이해

CODE · PR_FUNC_COMPARE.PY

```
from sklearn.metrics import precision_score, recall_score
print("함수 정밀도:", round(precision_score(y_test, y_pred), 4))
print("함수 재현율:", round(recall_score(y_test, y_pred), 4))
```

두 값이 일치하면 정밀도·재현율이 네 칸에서 어떻게 만들어지는지 완전히 이해한 상태

실습 — 손계산 체크포인트

분모 차이를 정확히 구분 — 정밀도는 **FP**, 재현율은 **FN**

01 손계산과 함수 결과가 일치하는가
공식과 함수가 같은 결과 — 정의를 완전히 이해

02 분모에 무엇이 들어가는지 구분했는가 (정밀도 **FP**, 재현율 **FN**)
분모의 차이가 두 지표의 본질적 차이

03 우리 모델은 정밀도·재현율 중 어느 쪽이 낮은가
재현율이 낮다면 고장을 놓치고 있다는 신호

실습 — 정밀도·재현율 적용과 설비 해석

정확도·정밀도·재현율을 한 화면에 — 치우침 판단과 현장 해석

목 표

정확도·정밀도·재현율을
한 화면에 **출력·비교**

세 지표를 나란히 두고 모델의 성격을 파악

핵 심

모델의 **치우침 판단**과
현장 해석

어느 쪽으로 치우쳤는지 현장 언어로 표현

실습 — 세 지표 한 화면 출력

정확도·정밀도·재현율을 함께 출력 — 정확도 높은데 재현율 낮은 식의 치우침 확인

CODE · THREE_METRICS.PY

```
from sklearn.metrics import accuracy_score, precision_score, recall_score
print("정확도:", round(accuracy_score(y_test, y_pred), 3))
print("정밀도:", round(precision_score(y_test, y_pred), 3))
print("재현율:", round(recall_score(y_test, y_pred), 3))
```

세 지표를 한 화면에 나란히 — 모델이 어느 쪽으로 치우쳤는지 한눈에 파악

실습 — 치우침 판단과 해석

"이 모델은 고장을 00% 잡고 경보 정확도는 00%" 한 문장으로 정리

패턴

해석

정확도↑ · 재현율↓

→ 불균형에 속은 모델 의심 — 다수 클래스만 잘 맞히는 중

재현율↑ · 정밀도↓

→ 고장은 잘 잡지만 헛경보 많음 — 적극적 모델

한 문장 정리 — "이 모델은 고장을 00% 잡고 경보 정확도는 00%"

실습 — 최종 정리 체크포인트

정확도의 한계부터 혼동행렬, 정밀도·재현율까지 — 분류 평가의 기초 완성

01 세 지표를 모두 구하고 비교했는가
정확도, 정밀도, 재현율 한 화면에 — 비교 준비 완료

02 모델의 치우침을 한 문장으로 설명했는가
수치를 의사결정 언어로 번역하는 능력

03 재현율을 우선해야 하는 이유를 말할 수 있는가
놓침이 가장 비싸기 때문 — 모듈 완전 익힘의 마지막 관문